Computer Networks (EE353)

Lab 5: TCP Traffic Analysis using Wireshark

Arqam Waheed 553514 BSDS 3A 8 October 2026

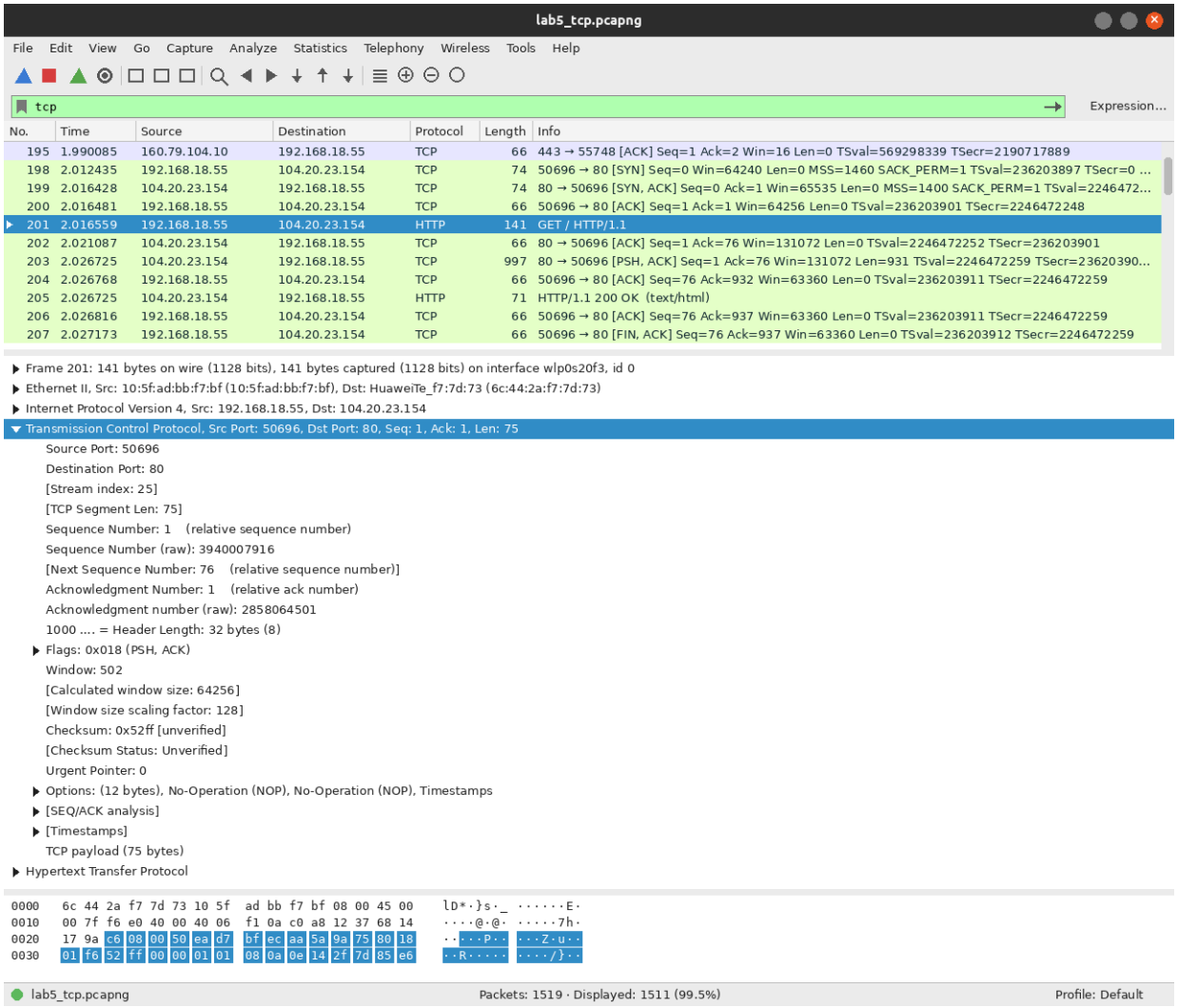
Setup

Wireshark 3.6.2 on Ubuntu, Wi-Fi interface wlp0s20f3 (my IP 192.168.18.55). Capture file lab5_tcp.pcapng (1519 packets, 1511 TCP). During the capture I opened example.com over HTTP and HTTPS with curl, then tried port 81 on my router to get a reset.

```
arqamwd@ArqamWd:~$ curl -4 -s -o /dev/null -w "http %{http_code}\n" --http1.1 http://example.com/
http 200
arqamwd@ArqamWd:~$ curl -4 -s -o /dev/null -w "https %{http_code}\n" https://example.com/
https 200
arqamwd@ArqamWd:~$ exec 3<>/dev/tcp/192.168.18.1/81
bash: connect: Connection refused
arqamwd@ArqamWd:~$
```

Commands used to generate TCP traffic

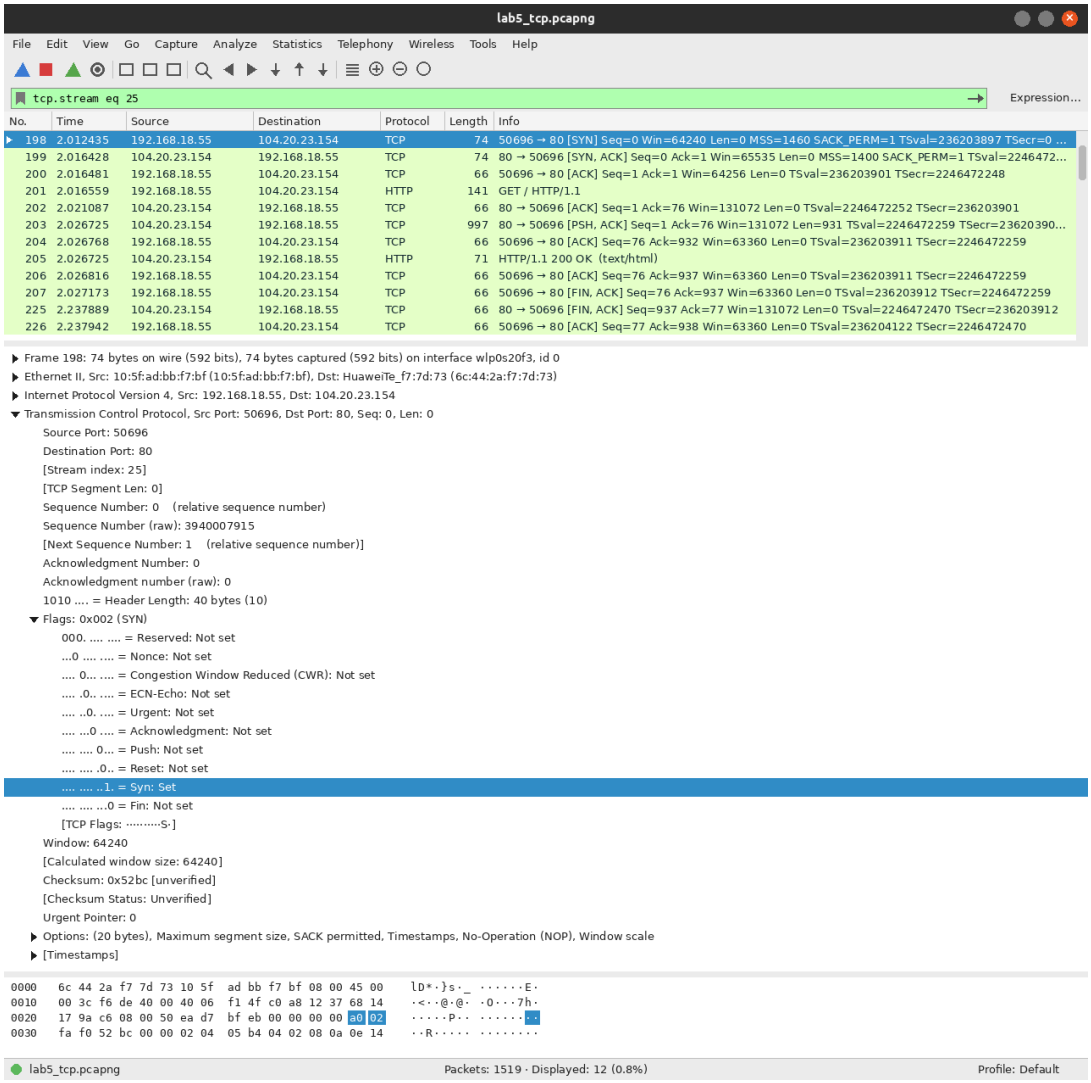
Task 1) Identify TCP Traffic



Filter: tcp, frame 201

Field	**Observation**
Source IP	192.168.18.55
Destination IP	104.20.23.154 (example.com)
Source Port	50696
Destination Port	80
Sequence Number	1 (raw 3940007916)
Acknowledgment Number	1 (raw 2858064501)
Flags	0x018 (PSH, ACK)
Window Size	502 × 128 = 64256 bytes
TCP Header Length	32 bytes (20 + 12 bytes of options)
Checksum / Status	0x52ff [unverified]
Higher Layer Protocol	HTTP (GET / HTTP/1.1)

Task 2) Find the Three-Way Handshake



Filter: tcp.stream eq 25, frames 198, 199, 200

Packet	Frame	Ports	Seq	Ack
SYN	198	50696 → 80	0 (raw 3940007915)	—
SYN/ACK	199	80 → 50696	0 (raw 2858064500)	1 (raw 3940007916)
ACK	200	50696 → 80	1	1 (raw 2858064501)

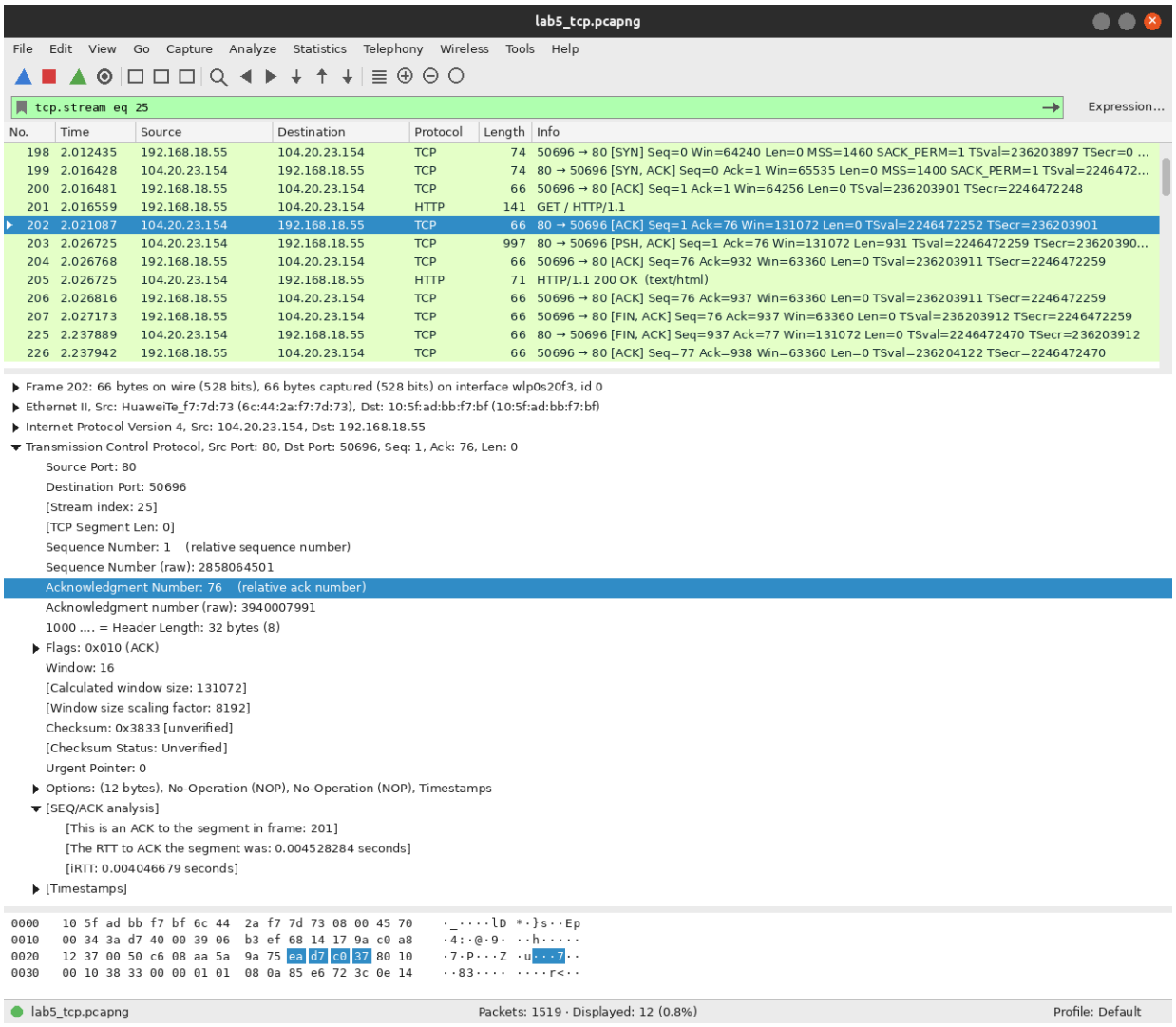
Purpose of SYN: It asks to open a connection and sends the client's starting sequence number (3940007915).

What SYN/ACK acknowledges: The client's SYN. Its Ack is the client's ISN + 1 (3940007916). It also carries the server's own ISN.

Why the final ACK: It confirms the server's SYN, so both sides know the other can send and receive. Then data can flow.

Who initiated: My laptop (192.168.18.55), because it sent the SYN.

Task 3) Sequence and Acknowledgment Analysis



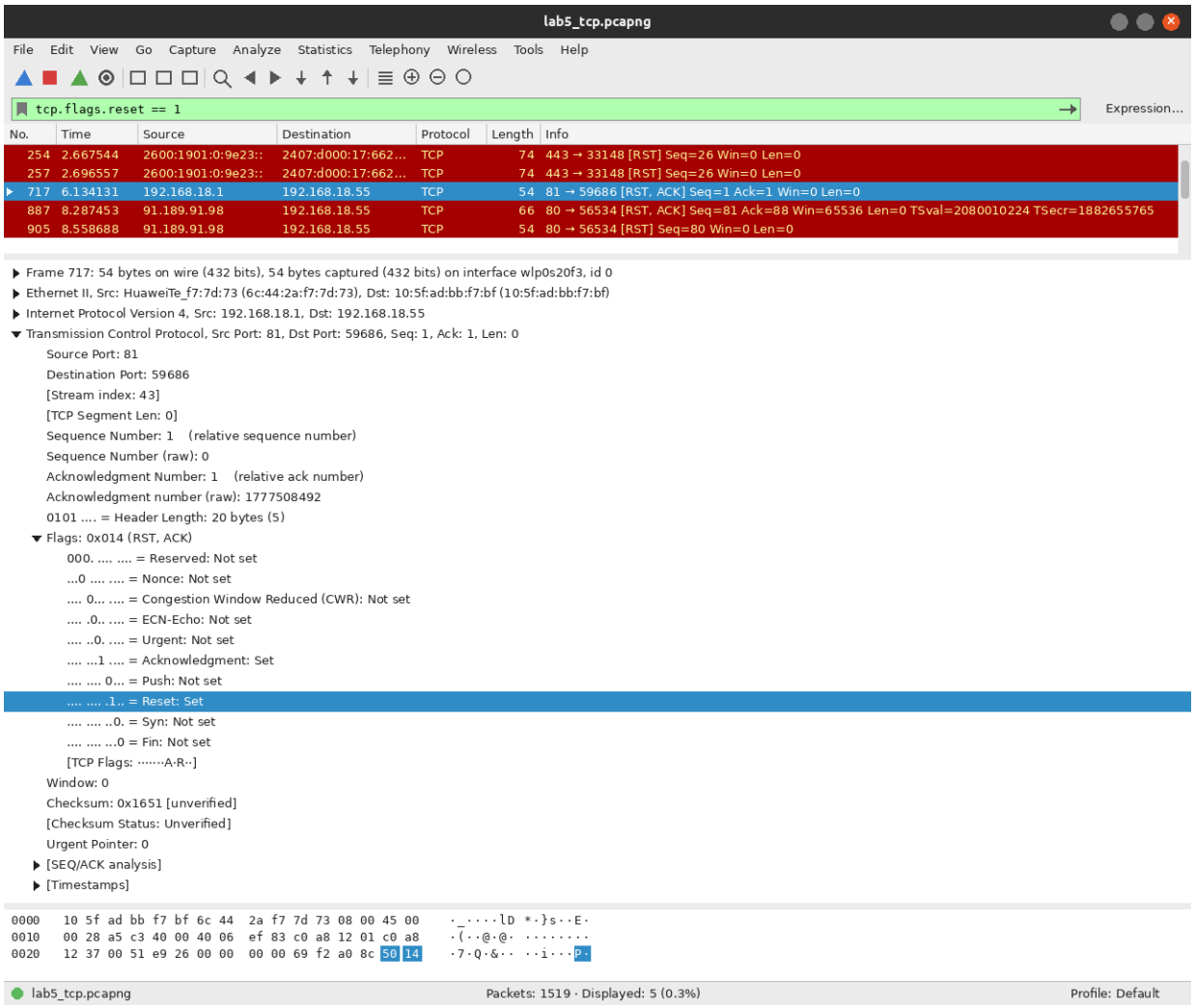
Filter: tcp.stream eq 25, ACK in frame 202 for data in frame 201

Field	Observation
Data packet	Frame 201 (HTTP GET), client → server
Sequence number	1
TCP segment length	75 bytes
Next sequence number	76 (1 + 75)
ACK packet	Frame 202, server → client
Acknowledgment number	76
Time between data and ACK	2.021087 − 2.016559 = 0.004528 s (4.5 ms)

How seq/ack give reliable, ordered delivery: Each byte has a sequence number, so the receiver can put segments back in order and spot gaps. The ACK number tells the sender the next byte expected (76 here means all 75 bytes arrived). Anything not acknowledged in time is resent. The SYN used up sequence number 0, which is why data starts at 1.

Task 4) Analyze TCP Flags

Filter	Packets	Meaning
tcp.flags.syn == 1	13	Opening a connection (SYN and SYN/ACK)
tcp.flags.ack == 1	1501	Ack number is valid; set on almost every packet after the SYN
tcp.flags.fin == 1	11	Sender has no more data and wants to close
tcp.flags.reset == 1	5	Connection aborted immediately



Filter: tcp.flags.reset == 1, frame 717

Frame 717 is the router (192.168.18.1) answering my SYN to port 81 with RST, ACK because nothing listens on that port.

Task 5) Follow TCP Stream



Analyze → Follow → TCP Stream on frame 201

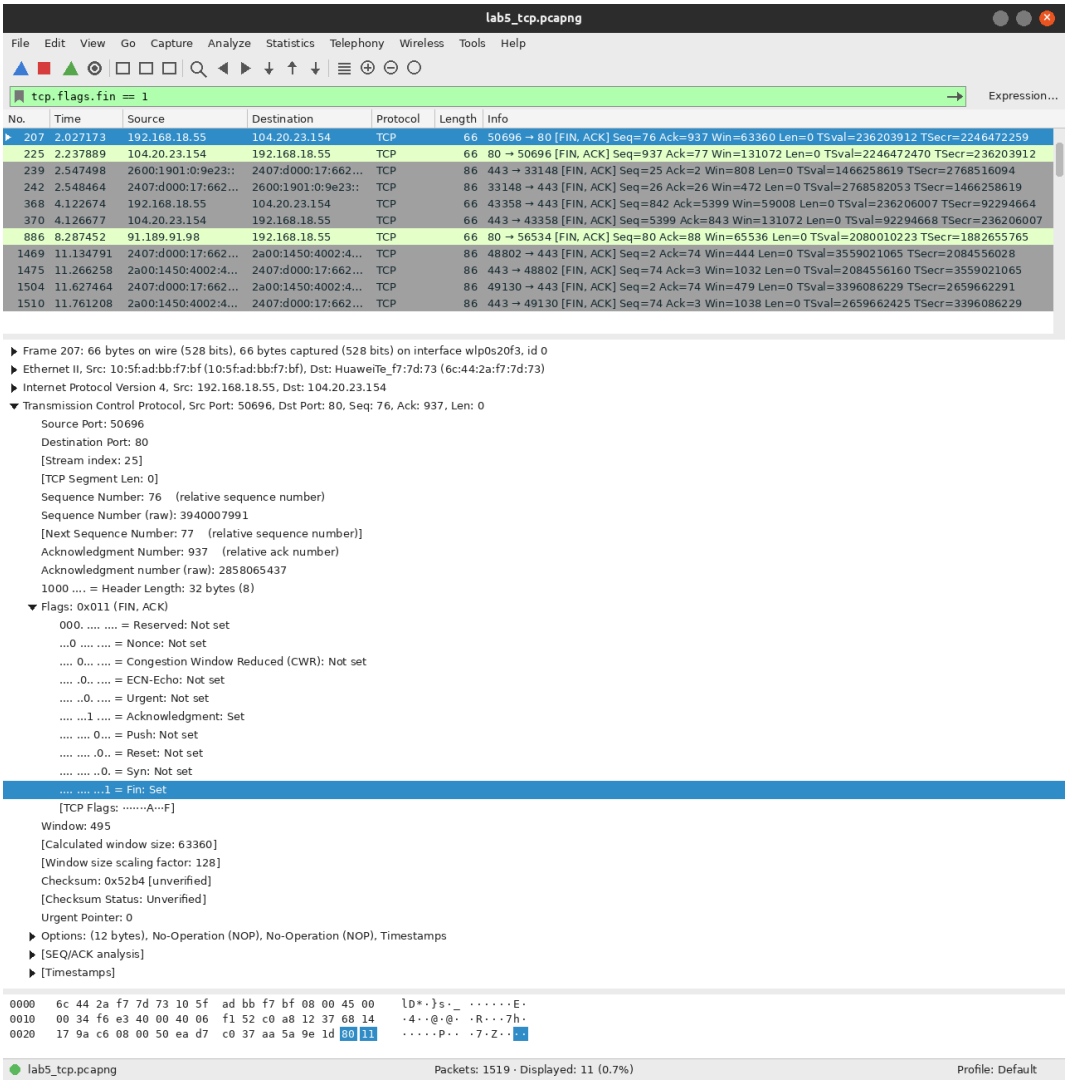
Stream 25. Client 192.168.18.55:50696, server 104.20.23.154:80. Wireshark applied tcp.stream eq 25, showing only the 12 packets of this connection. Red is what the client sent (GET request), blue is the server's reply (HTTP 200 OK).

What a TCP stream is: One connection, identified by the two IPs and two ports.

Why it is useful: It hides unrelated traffic so you can read one conversation from start to end.

Why HTTPS data is not readable: TLS encrypts the payload. Following the HTTPS stream (34) only shows random bytes.

Task 6) Observe Connection Termination



Filter: tcp.flags.fin == 1, frame 207

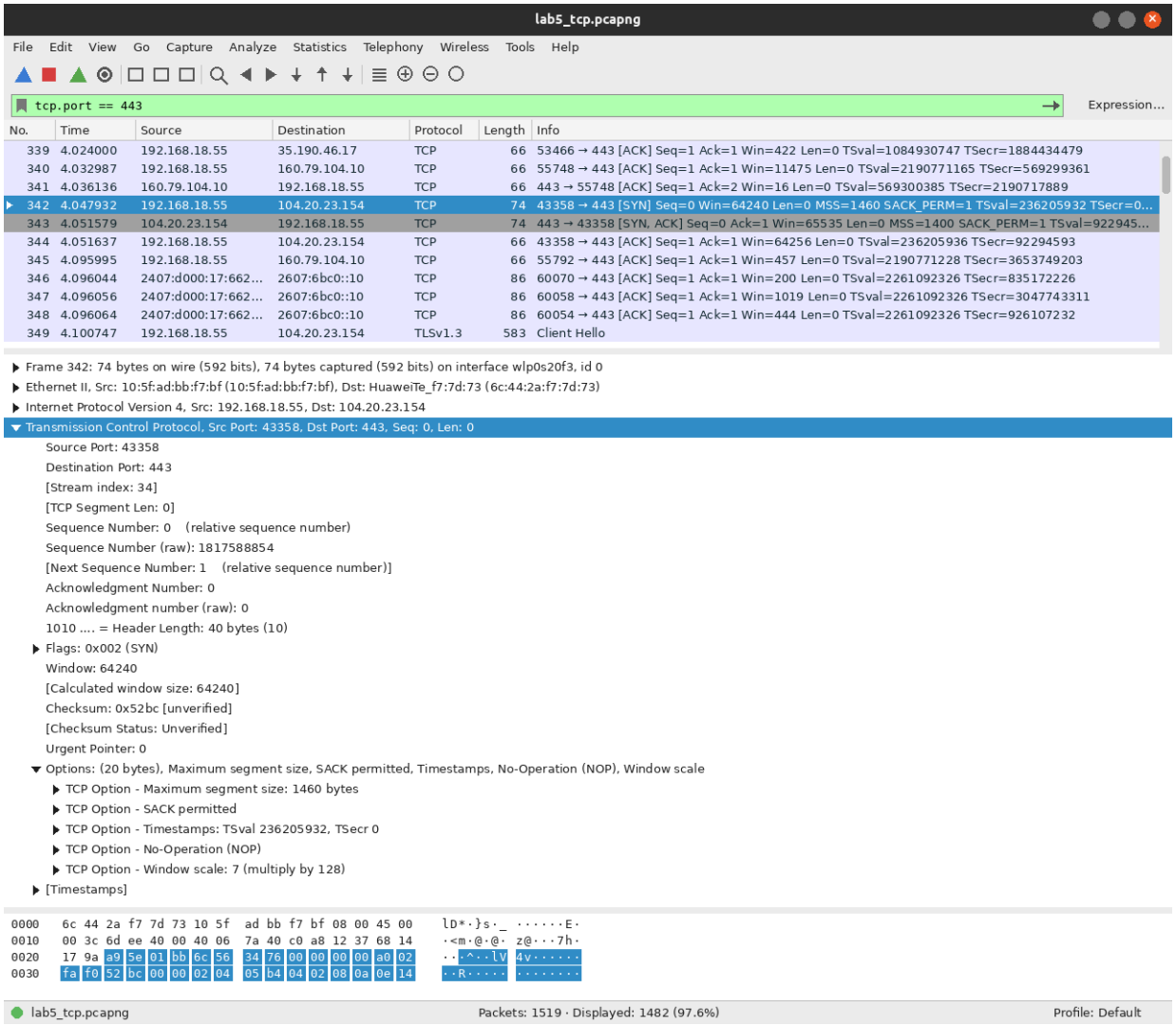
Frame	Direction	Flags	Seq / Ack
207	client → server	FIN, ACK	Seq 76, Ack 937
225	server → client	FIN, ACK	Seq 937, Ack 77
226	client → server	ACK	Seq 77, Ack 938

My client closed first. The server acknowledged the FIN and sent its own FIN in the same packet, so only 3 packets were needed. The HTTPS connection (frames 368–371) closed with the full 4 packets.

RST was also seen: frame 717 (closed port 81 on the router) and frames 887/905 where the Ubuntu connectivity-check server reset its connection.

Why multiple packets in both directions? TCP is full duplex, so each side closes its own direction with a FIN and the other side has to ACK it.

Task 7) Analyze TCP Port 443



Filter: tcp.port == 443, frame 342

1482 packets matched. Stream 34 (192.168.18.55:43358 → 104.20.23.154:443) did a normal handshake: SYN (342), SYN/ACK (343), ACK (344), then a TLS Client Hello in frame 349. Port 443 is normally HTTPS, but the port alone does not prove it. Wireshark only knows it is TLS because it dissected the payload.

Task 8) Compare TCP and UDP

Feature	TCP	UDP
Connection setup	Yes (3-way handshake)	No
Reliability	Yes (ACKs, retransmission)	No
Ordering	Yes (sequence numbers)	No
Acknowledgments	Yes	No
Flow control	Yes (receive window)	No
Minimum transport header	20 bytes	8 bytes
Examples	HTTP, HTTPS, SSH, email	DNS, DHCP, QUIC, video

- Here TCP needed 3 packets (198–200) before sending any data. In Lab 4 the DNS lookup was just a query and a reply.
- The TCP header in my packets was 32 bytes (20 + 12 bytes of timestamp options). The UDP header in Lab 4 was always 8 bytes.
- Frame 202 acknowledged the GET in frame 201. UDP had nothing like this.

Viva Questions

1. What is TCP and why is it connection-oriented? A reliable transport protocol. It sets up a connection with a handshake before sending data.

2. Three-way handshake? Client sends SYN, server replies SYN/ACK, client sends ACK.

3. Purpose of SYN? To start a connection and share the initial sequence number.

4. Sequence vs acknowledgment number? Seq is the number of the first byte in this segment. Ack is the next byte the sender expects to receive.

5. Why does TCP use ACKs? So the sender knows what arrived and can resend what didn't.

6. What is the receive window? How many bytes the receiver can still accept. The sender cannot have more than this unacknowledged.

7. Flow control vs congestion control? Flow control protects the receiver (window). Congestion control protects the network (slows down on loss).

8. FIN vs RST? FIN closes gracefully after all data is sent. RST aborts the connection immediately.

9. What does Follow TCP Stream do? Filters one connection (tcp.stream eq N) and shows its payload as a conversation.

10. Why might a website not produce TCP/443? The browser may use HTTP/3 (QUIC over UDP/443), or the page was cached.

11. What is a TCP segment? The TCP header plus the data it carries.

12. How does TCP differ from UDP? TCP is connection-oriented, reliable and ordered. UDP is connectionless with no guarantees.

13. Retransmission / duplicate ACK? Packet loss or reordering on the path. Several duplicate ACKs make the sender resend early.